NEXT ENGINEER · _SHEERAT_

Cloud Computing

BCS071

One-day revision notes: definitions, solved examples, code and important questions

Program: B.Tech | **Branches:** CSE, IT | **Semester:** 7 | **Credits:** 3 | **L-T-P:** 3-0-0

Scheme: AKTU B.Tech 4th year, Departmental Elective-IV, 2025-26 onwards

UNIT 1 Introduction to Cloud Computing

UNIT 2 Cloud Enabling Technologies and Virtualization

UNIT 3 Cloud Architecture, Services and Storage

UNIT 4 Resource Management and Security in Cloud

UNIT 5 Cloud Technologies and Advancements

Original revision notes written for the AKTU syllabus. Every unit has the same parts:

1. **2-mark definitions**: short answers you can write word for word in Section A.
2. **Core concepts**: models, architectures and comparison tables.
3. **Solved examples**: the calculations and scenarios that appear in the paper (speedup and Amdahl's law, availability, elasticity and autoscaling, cost comparison, MapReduce word count trace, VM placement).
4. **Code / key points to remember**.
5. **Important questions**: the long questions to prepare, with what your answer must contain.

How to use these notes the day before the paper

- **Morning (3 hours):** Unit 1 (definition, evolution, parallel and distributed computing, characteristics, elasticity, on-demand provisioning) and Unit 2 (SOA, REST, web services, publish-subscribe, virtualization types, levels, mechanisms, CPU, memory and I/O virtualization, disaster recovery).
- **Afternoon (3 hours):** Unit 3 (layered and NIST architecture, deployment models, IaaS, PaaS, SaaS, design challenges, storage, S3) and Unit 4 (inter-cloud resource management, provisioning methods, security challenges, IAM, VM security, standards).
- **Evening (2 hours):** Unit 5 (Hadoop, MapReduce, VirtualBox, Google App Engine, OpenStack, federation). Then the "Last-hour checklist".
- **In the exam:** draw the NIST reference architecture, the hypervisor types and the service model stack; they earn marks on their own.

AKTU paper pattern (theory, 70 marks, 3 hours): Section A has 7 short questions of 2 marks, all compulsory (14). Section B has 5 questions of 7 marks, attempt any 3 (21). Section C has 5 questions (one per unit), each with two parts, attempt one part of each, 7 marks each (35). Every unit gets its own Section C question, so do not skip any unit.

UNIT 1 Introduction to Cloud Computing

✍️ 2-mark definitions

- **Cloud computing (NIST SP 800-145):** a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (networks, servers, storage, applications and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction.
- **Elasticity:** the ability to grow or shrink resources automatically and quickly as demand changes, so the resources in use match the load at each moment.
- **Scalability:** the ability of a system to handle more load by adding resources; vertical (scale up, a bigger machine) or horizontal (scale out, more machines).
- **On-demand provisioning:** a user can obtain computing capacity (servers, storage) by themselves, whenever needed, without a human at the provider.
- **Parallel computing:** many processors working at the same time on parts of one problem, usually with shared memory and a tight coupling.
- **Distributed computing:** many independent computers connected by a network, each with its own memory, cooperating by passing messages.
- **Grid computing:** combining loosely coupled, geographically spread resources from several organizations to solve large problems (a virtual organization).
- **Utility computing:** offering computing resources as a metered service, like electricity, where you pay for what you use.
- **Multi-tenancy:** one instance of software or hardware serving many customers (tenants) whose data is kept isolated.
- **Pay-as-you-go:** billing based on actual usage (per hour, per GB) with no upfront capital cost.

💡 Core concepts

Evolution of cloud computing

Stage	Period	Key idea
Mainframes and time sharing	1960s–70s	Many users share one big computer through terminals
Client-server	1980s	PCs connect to central servers
Cluster computing	1990s	Many machines in one place act as one
Grid computing	late 1990s	Resources shared across organizations
Utility computing	2000s	Metered computing like a utility
Virtualization	2000s	Many virtual machines on one physical host (VMware, Xen)
Web services and SOA	2000s	Software as reusable services over the web

Stage	Period	Key idea
Cloud computing	2006 onwards	AWS EC2 and S3 (2006), Google App Engine (2008), Azure (2010)

Parallel and distributed computing principles

- **Flynn's taxonomy:** SISD (single processor), SIMD (one instruction on many data, GPUs, vector processors), MISD (rare), MIMD (multicore, clusters).
- **Memory models:** shared memory (all processors see one address space, threads, OpenMP) vs distributed memory (each node has its own memory, message passing, MPI).
- **Parallelism types:** bit-level, instruction-level, data parallelism (same operation on different data), task parallelism (different tasks at the same time).
- **Distributed system goals:** resource sharing, openness, concurrency, scalability, fault tolerance, transparency (access, location, migration, replication, failure).
- **Amdahl's law:** speedup = 1 / ((1 − f) + f / N), where f is the parallel fraction and N the number of processors. The serial part limits the speedup.

Parallel vs distributed computing

Feature	Parallel	Distributed
Location	One machine (multi-core) or tightly coupled cluster	Many machines over a network
Memory	Shared (mostly)	Distributed, each node private
Communication	Through shared memory or a bus	Message passing over the network
Goal	Speed up one computation	Share resources, reliability, scale
Clock	Often a common clock	No global clock
Example	GPU training, multi-threaded sort	Web search, cloud services, blockchain

Essential characteristics of cloud (NIST, five)

1. **On-demand self-service:** provision resources without human interaction with the provider.
2. **Broad network access:** available over the network through standard devices (phones, laptops).
3. **Resource pooling:** provider resources serve many consumers with multi-tenancy; location independence.
4. **Rapid elasticity:** capacity scales out and in quickly, often automatically, and seems unlimited.
5. **Measured service:** usage is metered, monitored and reported (pay per use).

Benefits: no capital expenditure (CapEx becomes OpEx), scalability, high availability, global reach, faster deployment, automatic updates, disaster recovery. **Drawbacks:** dependence on the Internet, security and privacy concerns, vendor lock-in, limited control, compliance and data location issues, unpredictable costs if not managed.

Elasticity in cloud

- Auto-scaling uses rules (for example: add one VM when average CPU > 70% for 5 minutes, remove one when below 30%) or predictive models.
- Horizontal elasticity adds or removes instances behind a load balancer; vertical elasticity resizes an instance.

- Elasticity vs scalability: scalability is the capacity to grow; elasticity is growing and shrinking automatically to match demand in real time.
- Over-provisioning wastes money; under-provisioning loses users. Elasticity avoids both.

On-demand provisioning: resources are requested through a portal, CLI or API (for example `aws ec2 run-instances`) and are ready in minutes. Provisioning types: advance reservation (reserved instances, cheaper, committed), on-demand (pay per hour, flexible), spot (spare capacity at a discount, can be reclaimed).

🧮 Solved examples

Example 1: Amdahl's law. 90% of a program can run in parallel. Speedup on 8 processors = 1 / (0.1 + 0.9/8) = 1 / (0.1 + 0.1125) = 1 / 0.2125 = 4.71. Maximum possible speedup with infinite processors = 1 / 0.1 = 10.

Example 2: Cost comparison. A startup needs 10 servers for 6 months. Buying: 10 × ₹2,00,000 = ₹20,00,000 plus power and staff. Cloud at ₹20 per server-hour: 10 × 20 × 24 × 180 = ₹8,64,000, and nothing to dispose of after 6 months. If demand is only 10 servers for 8 hours a day and 2 servers otherwise, cloud cost = 20 × 180 × (10 × 8 + 2 × 16) = 20 × 180 × 112 = ₹4,03,200, which shows the gain from elasticity.

Example 3: Auto-scaling. Rule: 1 VM handles 100 requests per second; keep CPU under 70%, so plan 70 requests per second per VM. Load rises from 300 to 1000 requests per second. VMs needed = ceil(1000 / 70) = 15 (from ceil(300/70) = 5), so the auto-scaler adds 10 VMs.

Key points to remember

- NIST definition; five characteristics; three service models; four deployment models.
- Elasticity = automatic grow and shrink; scalability = ability to grow.
- Amdahl: speedup = 1 / ((1 − f) + f/N); limit 1/(1 − f).
- Evolution: mainframe → client-server → cluster → grid → utility → virtualization → cloud (AWS 2006).

⭐ Important questions

1. Define cloud computing. Explain the evolution of cloud computing.
2. Explain the principles of parallel and distributed computing. Compare them.
3. Explain the essential characteristics of cloud computing. List its benefits and drawbacks.
4. What is elasticity in the cloud? How is it different from scalability? Explain auto-scaling.
5. Explain on-demand provisioning with its types.

NEXT ENGINEER _sheerat_

UNIT 2 Cloud Enabling Technologies and Virtualization

✍️ 2-mark definitions

- **SOA (Service-Oriented Architecture):** a design style in which an application is built from loosely coupled, reusable services that talk through standard interfaces over a network.
- **Web service:** a software component that can be called over the web using standard protocols; SOAP/WSDL/UDDI (classic) or REST/JSON.
- **REST (Representational State Transfer):** an architectural style that treats everything as a resource identified by a URI and manipulated with HTTP methods (GET, POST, PUT, DELETE), stateless, cacheable.
- **System of systems:** a large system made from independent, separately managed systems that cooperate to deliver more than each could alone (for example an air-traffic network).
- **Publish-subscribe model:** publishers send messages to topics without knowing receivers; subscribers receive messages of topics they registered for; a broker decouples them in time, space and synchronization.
- **Virtualization:** creating a virtual (software) version of a resource such as a server, storage, network or OS, so one physical resource looks like many, or many look like one.
- **Hypervisor (Virtual Machine Monitor, VMM):** the software layer that creates and runs virtual machines and shares the hardware among them.
- **Type 1 (bare-metal) hypervisor:** runs directly on hardware (VMware ESXi, Xen, Microsoft Hyper-V, KVM).
- **Type 2 (hosted) hypervisor:** runs as an application on a host OS (VirtualBox, VMware Workstation).
- **Full virtualization:** the guest OS runs unmodified; the hypervisor traps and emulates privileged instructions (binary translation or hardware assist).
- **Para-virtualization:** the guest OS is modified to call the hypervisor through hypercalls instead of using privileged instructions (Xen); faster but needs OS changes.
- **Live migration:** moving a running VM from one physical host to another with almost no downtime.

💡 Core concepts

SOA principles: loose coupling, service contract (interface), abstraction, reusability, autonomy, statelessness, discoverability (registry), composability. Roles: service provider, service requester, service registry (publish, find, bind).

SOAP vs REST

Feature	SOAP	REST
Type	Protocol	Architectural style
Format	XML only	JSON, XML, text

Feature	SOAP	REST
Transport	HTTP, SMTP, others	HTTP
Interface description	WSDL	OpenAPI (optional)
State	Can be stateful	Stateless
Performance	Heavier	Lighter, cacheable
Security	WS-Security built in	HTTPS, OAuth
Use	Banking, enterprise	Web and mobile APIs, cloud APIs

REST constraints: client-server, stateless, cacheable, uniform interface, layered system, code on demand (optional). HTTP mapping: GET read, POST create, PUT replace/update, PATCH partial update, DELETE remove. Status codes: 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found, 500 Server Error.

Publish-subscribe: topic-based or content-based. Benefits: decoupling, scalability, many-to-many delivery. Cloud examples: AWS SNS, Google Cloud Pub/Sub, Apache Kafka, MQTT.

Implementation levels of virtualization

Level	What is virtualized	Example
Instruction Set Architecture (ISA)	One ISA emulated on another	QEMU, Bochs
Hardware abstraction level	Virtual hardware for whole OSes	VMware, Xen, KVM
Operating system level	Isolated user-space containers sharing one kernel	Docker, LXC, OpenVZ
Library (API) level	API calls translated	WINE, vCUDA
Application level	Process-level VM	Java JVM, .NET CLR

Virtualization structures: (1) hypervisor architecture (Type 1, Type 2; micro-kernel like Xen's Dom0 vs monolithic like VMware ESX), (2) binary translation with full virtualization, (3) para-virtualization with a modified guest OS. **Host-based** virtualization puts the VMM above a host OS.

Types of virtualization

- Server (hardware) virtualization, desktop virtualization (VDI), application virtualization, storage virtualization (SAN pooling), network virtualization (VLAN, SDN, VXLAN), data virtualization, OS-level (containers).

Virtual machines vs containers

Feature	VM	Container
Isolation	Full guest OS per VM	Shares host kernel
Size	GBs	MBs
Start time	Minutes	Seconds
Overhead	Higher	Lower
Security isolation	Stronger	Weaker

Feature	VM	Container
Example	VMware, KVM	Docker

Virtualization of CPU, memory and I/O

- **CPU:** x86 has privileged and sensitive instructions; some sensitive instructions do not trap in user mode, which made x86 hard to virtualize (Popek and Goldberg requirement). Solutions: binary translation (VMware), para-virtualization (Xen), and hardware assist (Intel VT-x, AMD-V add a root mode for the VMM).
- **Memory:** two-level mapping: guest virtual → guest physical → machine (host) physical. The VMM keeps shadow page tables, or hardware does it with Extended/Nested Page Tables (Intel EPT, AMD NPT). Techniques: ballooning (reclaim guest memory), memory overcommit, page sharing (deduplication).
- **I/O:** full device emulation (slow, compatible), para-virtual drivers (virtio, split drivers in Xen with front end in guest and back end in Dom0), direct I/O / passthrough (SR-IOV, fastest).

Virtualization tools and mechanisms: VMware vSphere/ESXi, Citrix Xen/XenServer, KVM with QEMU, Microsoft Hyper-V, Oracle VirtualBox, Docker, Kubernetes for container orchestration.

Virtualization support and disaster recovery

- Benefits of virtualization: server consolidation, better utilization, isolation, fast provisioning from templates, snapshots, live migration, easier testing.
- Disaster recovery (DR): VMs are just files, so they can be replicated to another site; snapshots allow rollback; live migration avoids planned downtime; failover to a standby site. Measures: **RPO** (Recovery Point Objective, how much data loss is acceptable, in time) and **RTO** (Recovery Time Objective, how quickly service must be back).
- Live migration steps (pre-copy): reserve resources on the target, iteratively copy memory pages while the VM runs, stop the VM and copy the remaining dirty pages and CPU state, start on the target, release the source.

🧮 Solved examples

Example 1: Server consolidation. 40 physical servers run at 10% average CPU. Consolidate onto hosts with the same capacity, keeping each host at most 70%. Total load = 40 × 10% = 4 server-equivalents. Hosts needed = ceil(4 / 0.7) = 6. Saving = 34 servers (85%).

Example 2: REST design. A library service:

- `GET /books` lists books; `GET /books/42` reads book 42.
- `POST /books` with JSON creates a book (201 Created).
- `PUT /books/42` replaces it; `DELETE /books/42` removes it (204 No Content).

Example 3: Live migration time (pre-copy). VM memory 4 GB, link 1 GB/s, dirty rate 0.2 GB/s. Round 1 copies 4 GB in 4 s, during which 0.8 GB gets dirty. Round 2 copies 0.8 GB in 0.8 s, 0.16 GB dirty. Round 3 copies 0.16 GB in 0.16 s, 0.032 GB dirty. Stop-and-copy of 0.032 GB takes 0.032 s, so downtime is about 32 ms, and total migration time is about 4.99 s.

NEXT ENGINEER _sheerat_

Key points to remember

- SOA roles: provider, requester, registry; publish, find, bind.
- REST: resources, URIs, HTTP verbs, stateless.
- Type 1 on bare metal (ESXi, Xen, Hyper-V, KVM); Type 2 on a host OS (VirtualBox).
- Full virtualization (unmodified guest) vs para-virtualization (modified guest, hypercalls).
- Five levels: ISA, hardware, OS, library, application.
- Memory: shadow page tables or EPT/NPT; I/O: emulation, para-virtual, passthrough.
- RPO = data loss tolerance; RTO = downtime tolerance.

⭐ Important questions

1. Explain SOA and its principles. Compare SOAP and REST web services.
2. What is the publish-subscribe model? Explain its role in the cloud.
3. What is virtualization? Explain the levels of virtualization implementation.
4. Explain Type 1 and Type 2 hypervisors. Compare full and para-virtualization.
5. Explain the virtualization of CPU, memory and I/O devices.
6. Explain virtualization support for disaster recovery and live migration.

NEXT ENGINEER _sheerat_

UNIT 3 Cloud Architecture, Services and Storage

✍️ 2-mark definitions

- **IaaS (Infrastructure as a Service):** the provider supplies virtual machines, storage and networks; the user manages the OS, runtime and applications (AWS EC2, Google Compute Engine, Azure VMs).
- **PaaS (Platform as a Service):** the provider supplies a platform (runtime, middleware, database) to build and deploy applications without managing servers (Google App Engine, Heroku, Azure App Service).
- **SaaS (Software as a Service):** complete applications delivered over the Internet, used through a browser (Gmail, Salesforce, Microsoft 365).
- **Public cloud:** infrastructure owned by a provider and offered to the general public over the Internet.
- **Private cloud:** cloud infrastructure used by one organization only, on or off premises.
- **Hybrid cloud:** a combination of public and private clouds bound by technology that allows data and application portability (cloud bursting).
- **Community cloud:** shared by several organizations with common concerns (for example government departments).
- **Cloud bursting:** running normally in a private cloud and "bursting" to a public cloud when demand peaks.
- **Storage as a Service (STaaS):** renting storage capacity from a provider over the network, paying per GB.
- **Amazon S3 (Simple Storage Service):** an object storage service where data is stored as objects inside buckets, accessed by key through a REST API, with 99.999999999% (11 nines) durability.

💡 Core concepts

Layered cloud architecture

Layer	Contents	Service model
Application layer	End-user applications	SaaS
Platform layer	OS, runtime, middleware, frameworks	PaaS
Infrastructure layer	Virtualized compute, storage, network	IaaS
Hardware (data center) layer	Physical servers, switches, power, cooling	Physical resources

Who manages what

Item	On-premises	IaaS	PaaS	SaaS
Applications	You	You	You	Provider
Data	You	You	You	Provider (you own it)

Item	On-premises	IaaS	PaaS	SaaS
Runtime and middleware	You	You	Provider	Provider
OS	You	You	Provider	Provider
Virtualization	You	Provider	Provider	Provider
Servers, storage, network	You	Provider	Provider	Provider

NIST cloud computing reference architecture: five actors

1. **Cloud consumer:** uses the services.
2. **Cloud provider:** makes services available; does service orchestration (service layer SaaS/PaaS/IaaS, resource abstraction and control layer, physical resource layer), cloud service management (business support, provisioning and configuration, portability and interoperability), security and privacy.
3. **Cloud auditor:** independently assesses services for security, privacy and performance.
4. **Cloud broker:** manages the use, performance and delivery of services and negotiates between providers and consumers (service intermediation, aggregation, arbitrage).
5. **Cloud carrier:** provides connectivity and transport between consumers and providers (network operators).

Deployment models compared

Feature	Public	Private	Hybrid	Community
Owner	Provider	One organization	Mix	Group of organizations
Cost	Low, pay per use	High upfront	Medium	Shared
Security and control	Lower	Highest	Balanced	Medium
Scalability	Very high	Limited by hardware	High	Medium
Example	AWS, Azure, GCP	OpenStack in a bank	Bank core private, web public	Government cloud (MeghRaj)

Architectural design challenges

- Service availability and data lock-in (use multiple providers, standard APIs).
- Data privacy and security concerns.
- Unpredictable performance and bottlenecks (I/O sharing, network).
- Distributed storage and widespread software bugs (debugging at scale).
- Cloud scalability, interoperability and standardization.
- Software licensing and reputation sharing (one tenant's bad behaviour affects others).
- Data transfer bottlenecks and costs.

Cloud storage

Type	Description	Example
Object storage	Flat namespace, objects with metadata, REST API, huge scale	Amazon S3, Azure Blob, Google Cloud Storage
Block storage	Raw volumes attached to VMs, low latency	Amazon EBS, Azure Disk
File storage	Shared file system with folders (NFS/SMB)	Amazon EFS, Azure Files
Database storage	Managed databases	Amazon RDS, DynamoDB, Cloud SQL

Advantages of cloud storage: pay per use, unlimited scale, durability through replication, access from anywhere, automatic backup and versioning, no hardware maintenance. **Disadvantages:** depends on Internet, latency, security and compliance concerns, egress costs.

Cloud storage providers: Amazon S3, Google Cloud Storage, Azure Blob Storage, Dropbox, Google Drive, Box, IBM Cloud Object Storage.

Amazon S3 in detail

- Concepts: **bucket** (container with a globally unique name, in a region), **object** (data + metadata, up to 5 TB), **key** (unique name of an object in a bucket).
- Access: REST API, SDKs, CLI; URLs like `https://bucket.s3.region.amazonaws.com/key`.
- Storage classes: S3 Standard, Standard-IA (infrequent access), One Zone-IA, Intelligent-Tiering, Glacier and Glacier Deep Archive (archive).
- Features: versioning, lifecycle rules (move to Glacier after 30 days), encryption at rest and in transit, bucket policies and IAM, static website hosting, cross-region replication, event notifications.
- Durability 99.999999999% by storing copies across multiple Availability Zones.

🧮 Solved examples

Example 1: Choosing a service model.

- A student wants a ready email service: SaaS (Gmail).
- A developer wants to deploy a Python web app without managing servers: PaaS (App Engine).
- A company wants full control over the OS to run legacy software: IaaS (EC2).

Example 2: Availability. A service is 99.9% available. Downtime per year = 0.001 × 365 × 24 = 8.76 hours. At 99.99%: 52.6 minutes. Two independent replicas each 99% available, with failover: availability = 1 − (0.01)^2 = 99.99%. Two components in series, each 99.9%: 0.999^2 = 99.8%.

Example 3: Storage cost. Store 500 GB for a year. S3 Standard at ₹1.9 per GB-month: 500 × 1.9 × 12 = ₹11,400. If 400 GB is rarely used and moved to an archive class at ₹0.3 per GB-month: 100 × 1.9 × 12 + 400 × 0.3 × 12 = 2,280 + 1,440 = ₹3,720. (Prices are illustrative; the method is what is examined.)

Code

S3 with the AWS CLI

```
aws s3 mb s3://my-notes-bucket-2026            # make bucket
aws s3 cp notes.pdf s3://my-notes-bucket-2026/  # upload object
aws s3 ls s3://my-notes-bucket-2026/            # list objects
aws s3 rm s3://my-notes-bucket-2026/notes.pdf   # delete object
```

Key points to remember

- SPI stack: SaaS (use), PaaS (build), IaaS (host).
- NIST actors: consumer, provider, auditor, broker, carrier.
- Deployment models: public, private, hybrid, community.
- Storage types: object (S3), block (EBS), file (EFS).
- S3: bucket, object, key, 5 TB max object, 11 nines durability, storage classes.
- 99.9% availability = 8.76 h downtime a year.

⭐ Important questions

1. Explain the layered cloud architecture design.
2. Explain the NIST cloud computing reference architecture with a diagram.
3. Explain public, private, hybrid and community clouds with merits and demerits.
4. Explain IaaS, PaaS and SaaS with examples. Who manages what?
5. Explain the architectural design challenges of cloud computing.
6. What is cloud storage? Explain Storage as a Service and Amazon S3.

NEXT ENGINEER _sheerat_

UNIT 4 Resource Management and Security in Cloud

✍️ 2-mark definitions

- **Inter-cloud (cloud of clouds):** interconnection of multiple clouds so that workloads and data can move between them.
- **Resource provisioning:** allocating cloud resources (VMs, storage, bandwidth) to applications to meet their QoS while keeping costs low.
- **SLA (Service Level Agreement):** a contract between provider and customer stating the service level (availability, response time) and penalties.
- **Global exchange of cloud resources:** a market-oriented system in which cloud providers and brokers trade resources, with a cloud exchange matching demand and supply (Buyya's model).
- **Cloud governance:** policies and processes for controlling cloud use: who may use what, cost control, compliance and risk.
- **IAM (Identity and Access Management):** the framework of policies and tools that ensures the right users have the right access to the right resources (users, groups, roles, policies).
- **Shared responsibility model:** the provider secures the cloud (hardware, facilities, hypervisor) and the customer secures what they put in it (data, access, configuration, OS in IaaS).
- **VM escape:** an attack in which code in a guest VM breaks out and runs on the hypervisor or another VM.
- **Multi-factor authentication (MFA):** requiring two or more independent factors (password + OTP).

💡 Core concepts

Inter-cloud resource management

- Extended cloud stack: IaaS, PaaS, SaaS plus Communication as a Service (CaaS) and Data as a Service (DaaS).
- Runtime support services: cluster monitoring, job management, scheduling, load balancing.
- Resource management across clouds: brokers discover resources, negotiate SLAs and place workloads in the cheapest or nearest suitable cloud.

Resource provisioning methods

Method	How it works	Pros and cons
Demand-driven	Add or remove resources when utilization crosses thresholds (auto-scaling)	Simple; reacts late to sudden spikes
Event-driven	Pre-provision for known events (festival sale, exam results)	Good for predictable peaks; needs prediction
Popularity-driven	Provision based on popularity of the service (Internet search trends)	Anticipates demand; may waste resources if wrong

Method	How it works	Pros and cons
Static (over-provisioning)	Provision for peak always	Simple; wasteful

Problems: under-provisioning (SLA violation, lost users) and over-provisioning (wasted money). Also VM placement (bin packing), dynamic migration and power-aware consolidation.

Global exchange of cloud resources (market-oriented cloud)

- Actors: users (via brokers), cloud coordinators at each provider (publish offers, monitor, schedule), cloud exchange (market directory, auctioneer, bank for payments).
- Pricing: fixed, spot or auction based.
- Benefit: users get the best price and QoS; providers sell idle capacity.

Security overview

Concern	Detail
Data breaches	Misconfigured storage (public S3 buckets), weak access control
Data loss	Accidental deletion, no backup
Account hijacking	Stolen credentials, phishing
Insecure APIs	Poorly protected management interfaces
Insider threats	Malicious staff at provider or customer
Multi-tenancy	Side-channel attacks, VM escape
Denial of service	Exhausting shared resources
Compliance	Data location laws, audits
Loss of control	Customer cannot inspect provider infrastructure

Cloud security challenges: trust, multi-tenancy isolation, data location and jurisdiction, data remanence (deleted data still on disks), transparency and audit, key management, securing virtual networks.

SaaS security: data security and segregation per tenant, access control and SSO, authentication, encryption, web application security (OWASP Top 10: injection, broken authentication, XSS), availability, backup, identity management, patch management.

Security governance: security policy, risk assessment, roles and responsibilities, compliance (ISO 27001, SOC 2, GDPR, India's DPDP Act 2023), auditing, incident response plan, vendor management.

Virtual machine security

- Threats: VM escape, VM sprawl (unmanaged VMs), VM hopping (attack one VM from another), insecure VM images, rogue snapshots, hypervisor vulnerabilities, inter-VM traffic not monitored.
- Controls: harden and patch the hypervisor, minimal trusted images, virtual firewalls and security groups, VM introspection-based IDS, encrypted disks and migration, isolation of management network, monitoring.

Identity and access management

- Components: identity (users, groups, roles), authentication (password, MFA, certificates), authorization (policies, RBAC, ABAC), auditing (logs).
- Federation and SSO: SAML 2.0, OAuth 2.0 (authorization), OpenID Connect (authentication on top of OAuth).
- Principle of least privilege; separation of duties; rotate keys; avoid root account use.
- AWS IAM example: a policy JSON allows `s3:GetObject` on one bucket only, attached to a role used by an EC2 instance.

Security standards: ISO/IEC 27001 (ISMS), ISO/IEC 27017 (cloud controls), ISO/IEC 27018 (personal data in public cloud), CSA Cloud Controls Matrix and STAR, NIST SP 800-144/145, SOC 1/2/3 reports, PCI DSS (card data), HIPAA (health), SAML, OAuth, OpenID, SSL/TLS.

🧮 Solved examples

Example 1: Demand-driven provisioning. Thresholds: scale out if CPU > 80%, scale in if CPU < 30%. Four VMs at 90% CPU (total load 3.6 VM-units). After adding one VM: 3.6 / 5 = 72% (stops scaling). Later the load falls to 1.0 VM-unit: with 5 VMs, 20% (below 30%), remove one: 1.0/4 = 25%, still below, remove one: 1.0/3 = 33.3%, stops. Final: 3 VMs.

Example 2: VM placement (first-fit decreasing). Hosts have 16 GB RAM. VMs need 10, 8, 6, 5, 4, 3 GB. Sort decreasing and place each in the first host with room:

- Host 1: 10 + 6 = 16.
- Host 2: 8 + 5 + 3 = 16.
- Host 3: 4.

Three hosts are used (the lower bound is ceil(36/16) = 3, so it is optimal).

Example 3: Shared responsibility. In IaaS the customer patches the guest OS; in PaaS the provider patches the OS and runtime and the customer secures the code and data; in SaaS the customer only manages users, access and their data.

Code

A least-privilege IAM policy (JSON)

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["s3:GetObject"],
    "Resource": "arn:aws:s3:::my-notes-bucket-2026/*"
  }]
}
```

Key points to remember

- Provisioning methods: demand-driven, event-driven, popularity-driven.
- Global exchange: broker, coordinator, exchange (directory, auctioneer, bank).
- Shared responsibility: provider secures the cloud, customer secures what is in the cloud.

NEXT ENGINEER _sheerat_

- VM threats: escape, sprawl, hopping, insecure images.
- IAM: users, groups, roles, policies, MFA, least privilege, SAML/OAuth/OIDC.
- Standards: ISO 27001/27017/27018, CSA STAR, SOC 2, PCI DSS.

⭐ Important questions

1. Explain inter-cloud resource management.
2. Explain resource provisioning and the methods of provisioning: demand-driven, event-driven and popularity-driven.
3. Explain the global exchange of cloud resources.
4. Explain the security challenges in cloud computing. Explain SaaS security.
5. Explain virtual machine security and security governance.
6. Explain identity and access management and cloud security standards.

NEXT ENGINEER _sheerat_

UNIT 5 Cloud Technologies and Advancements

✍️ 2-mark definitions

- **Hadoop:** an open-source Apache framework for storing (HDFS) and processing (MapReduce, YARN) very large data sets across clusters of commodity machines.
- **HDFS:** Hadoop Distributed File System; splits files into blocks (128 MB by default), stores 3 replicas, with a NameNode (metadata) and DataNodes (data).
- **MapReduce:** a programming model in which a map function turns input into key-value pairs and a reduce function combines all values of the same key; the framework handles splitting, shuffling, sorting and fault tolerance.
- **YARN:** Yet Another Resource Negotiator, the Hadoop 2 resource manager (ResourceManager, NodeManager, ApplicationMaster).
- **VirtualBox:** a free, open-source Type 2 hypervisor from Oracle that runs guest OSes (Linux, Windows) on a host OS.
- **Google App Engine (GAE):** Google's PaaS for building and hosting web applications that scale automatically, with managed runtimes and services (Datastore, Memcache, Task Queues).
- **OpenStack:** an open-source cloud operating system to build public and private IaaS clouds, made of services such as Nova, Swift, Cinder, Neutron, Keystone, Glance and Horizon.
- **Cloud federation:** a set of cloud providers that interconnect voluntarily to share resources, so they can serve users beyond their own capacity or location.

💡 Core concepts

Hadoop architecture

- **HDFS:** NameNode (master, file namespace and block map), DataNodes (store blocks, send heartbeats), Secondary NameNode (checkpoints metadata). Write once, read many; rack-aware replica placement (one replica local, two on another rack).
- **MapReduce (Hadoop 1):** JobTracker (master) and TaskTrackers (workers). **Hadoop 2:** YARN separates resource management from processing.
- **Ecosystem:** Hive (SQL), Pig (scripts), HBase (NoSQL), Sqoop (import from RDBMS), Flume (log ingest), Oozie (workflow), ZooKeeper (coordination), Spark (in-memory processing).

MapReduce flow: input splits → map → combine (optional local reduce) → partition → shuffle and sort → reduce → output in HDFS.

VirtualBox: features: snapshots, shared folders, guest additions, virtual networking modes (NAT, bridged, host-only, internal), import/export OVF appliances, cross-platform. Steps: install, create VM (name, OS type, RAM, virtual disk VDI), attach ISO, install guest OS, install guest additions.

Google App Engine

- Runtimes: Python, Java, Go, PHP, Node.js, Ruby; standard environment (sandboxed, scales to zero, fast) and flexible environment (Docker containers on VMs).

- Programming environment: SDK, `app.yaml` config, local dev server, `gcloud app deploy`; services: Cloud Datastore/Firestore (NoSQL), Memcache, Task Queues, Cron, Cloud Storage, users and authentication.
- Advantages: no server management, automatic scaling and load balancing, pay per use, built-in versioning and traffic splitting. Limitations: sandbox restrictions, vendor lock-in.

OpenStack components

Service	Code name	Function
Compute	Nova	Creates and manages VMs
Object storage	Swift	S3-like object store
Block storage	Cinder	Volumes for VMs
Networking	Neutron	Virtual networks, IPs, routers
Identity	Keystone	Authentication and authorization
Image	Glance	VM images
Dashboard	Horizon	Web UI
Orchestration	Heat	Templates to deploy stacks
Telemetry	Ceilometer	Metering for billing

Federation in the cloud

- Motivation: no single provider has data centres everywhere or unlimited capacity; federation gives wider reach, overflow capacity, lower cost, avoids lock-in, and meets data location laws.
- **Four levels of federation** (Rajkumar Buyya / Mastering Cloud Computing): 1. **Conceptual level:** the federation as seen by users and providers: motivations, benefits, functional and non-functional requirements (trust, SLAs). 2. **Logical and operational level:** policies and rules for operating the federation: who joins, how resources are shared, market and pricing models, SLA management, the cloud exchange. 3. **Infrastructural level:** the technologies that make it work: interoperability standards, APIs, protocols, middleware, brokers. 4. **Applications level** (sometimes counted with the infrastructural level): applications built to run across federated clouds.
- **Federated services and applications:** federated identity (SSO across clouds), federated storage, cross-cloud load balancing, content delivery, scientific workflows across clouds.
- **Future of federation:** multi-cloud and hybrid tools (Kubernetes, Terraform), standards (OCCI, CDMI, TOSCA), edge and fog federation, AI-driven brokers, sovereign clouds and regulation-aware placement.

🧮 Solved examples

Example 1: MapReduce word count trace. Input split 1: "cloud data cloud"; split 2: "data cloud big".

- Map 1 emits (cloud,1), (data,1), (cloud,1). Map 2 emits (data,1), (cloud,1), (big,1).
- Shuffle and sort groups: big → [1]; cloud → [1,1,1]; data → [1,1].
- Reduce outputs: (big,1), (cloud,3), (data,2).
- With a combiner, map 1 sends (cloud,2), (data,1), reducing network traffic from 6 pairs to 5.

Example 2: HDFS storage. A 1 GB file with 128 MB blocks gives 1024 / 128 = 8 blocks. With replication 3, there are 24 block copies and 3 GB of raw storage used. A 300 MB file gives 3 blocks (128 + 128 + 44 MB; the last block uses only 44 MB on disk).

Code

Word count in Hadoop (Java, core parts)

```
public class WordCount {
    public static class TokenMapper
            extends Mapper<Object, Text, Text, IntWritable> {
        private final static IntWritable one = new IntWritable(1);
        private final Text word = new Text();
        public void map(Object key, Text value, Context ctx)
                throws IOException, InterruptedException {
            for (String w : value.toString().split("\\s+")) {
                word.set(w);
                ctx.write(word, one);
            }
        }
    }
    public static class SumReducer
            extends Reducer<Text, IntWritable, Text, IntWritable> {
        public void reduce(Text key, Iterable<IntWritable> vals, Context ctx)
                throws IOException, InterruptedException {
            int sum = 0;
            for (IntWritable v : vals) sum += v.get();
            ctx.write(key, new IntWritable(sum));
        }
    }
}
```

Minimal Google App Engine app (Python, Flask)

```
# app.yaml
runtime: python312
```

```
# main.py
from flask import Flask
app = Flask(__name__)

@app.route("/")
def hello():
    return "Hello from App Engine"
```

Deploy with `gcloud app deploy`.

Key points to remember

- HDFS: NameNode, DataNode, 128 MB blocks, replication 3.
- MapReduce: map → shuffle/sort → reduce; combiner reduces traffic.

- VirtualBox: Type 2 hypervisor, snapshots, NAT/bridged/host-only networks.
- GAE: PaaS, auto-scaling, standard vs flexible environment, `app.yaml`.
- OpenStack: Nova, Swift, Cinder, Neutron, Keystone, Glance, Horizon, Heat.
- Federation four levels: conceptual, logical and operational, infrastructural, applications.

⭐ Important questions

1. Explain Hadoop architecture: HDFS, MapReduce and YARN.
2. Explain the MapReduce programming model with the word count example.
3. Explain VirtualBox and how a VM is created in it.
4. Explain Google App Engine and its programming environment.
5. Explain OpenStack and its components with a diagram.
6. What is cloud federation? Explain the four levels of federation, federated services and the future of federation.

Last-hour checklist

☐ NIST definition; evolution table; five characteristics; benefits and drawbacks.

☐ Parallel vs distributed table; Flynn; Amdahl (4.71 speedup); elasticity vs scalability; provisioning types.

☐ SOA roles; SOAP vs REST table; REST verbs; publish-subscribe.

☐ Virtualization levels; Type 1 vs Type 2; full vs para; VM vs container; CPU, memory, I/O virtualization; live migration; RPO and RTO.

☐ Layered architecture; who-manages-what table; NIST five actors; deployment models table; design challenges.

☐ Storage types; S3 bucket/object/key, classes, 11 nines; availability numbers (8.76 h).

☐ Inter-cloud; provisioning methods; global exchange; security concerns table; SaaS security; governance.

☐ VM security threats; IAM components; shared responsibility; standards list.

☐ Hadoop, HDFS, YARN; word count trace; VirtualBox; GAE; OpenStack table; federation four levels.

Books for this subject

The AKTU syllabus copy we could access did not show the book list for this elective, so these are the standard textbooks that cover its units. Check your college's copy of the syllabus for the official list. Buy or borrow them through a bookshop or your college library. The WorldCat links show which libraries hold a copy.

#	Title	Author(s)	Publisher	Where to get it legally
1	Distributed and Cloud Computing: From Parallel Processing to the Internet of Things	Kai Hwang, Geoffrey C. Fox, Jack J. Dongarra	Morgan Kaufmann / Elsevier	WorldCat
2	Mastering Cloud Computing	Rajkumar Buyya, Christian Vecchiola, S. Thamarai Selvi	McGraw Hill Education	WorldCat
3	Cloud Computing: Implementation, Management, and Security	John W. Rittinghouse, James F. Ransome	CRC Press	WorldCat
4	Cloud Computing: A Practical Approach	Toby Velte, Anthony Velte, Robert Elsenpeter	McGraw Hill	WorldCat
5	Cloud Security and Privacy	Tim Mather, Subra Kumaraswamy, Shahed Latif	O'Reilly Media	WorldCat

The NIST definition of cloud computing (SP 800-145) and reference architecture (SP 500-292) are free government publications at csrc.nist.gov and nist.gov.

Syllabus source: AKTU B.Tech CS/CE and CSE 4th year syllabus, 2025-26 (copy hosted by an affiliated college, PDF), with the elective unit list cross-checked against aktunotes.in.

NEXT ENGINEER _sheerat_